Numbers, Booleans & Operators

NUMBERS

Types & literals

```
42  1_000_000  0xFF  0o17  0b1010  # int
3.14  1e3  2.5e-3   # float (1e3 is a float)
3+4j                # complex
True  False         # bool, a subclass of int
type(x)    isinstance(x, int)
007                 # SyntaxError
```

Wider type wins: bool → int → float → complex. `int / int` is always a float.

Convert

```
int("42")         → 42
int(3.9)          → 3      # truncates
int(-3.7)         → -3
int("ff", 16)     → 255
int("0b101", 0)   → 5      # auto base
int("3.5")        → ValueError
float(" 3.14 ")   → 3.14
```

Integers

```
2 ** 100            # never overflows
divmod(-7, 2)       → (-4, 1)
pow(2, 10, 1000)    → 24    # modular
bin(10)  oct(10)  hex(255)
                    → '0b1010' '0o12' '0xff'
format(10, "08b")   → '00001010'
(255).bit_length()  → 8
(12).bit_count()    → 2    # 3.10+
math.gcd(12, 18)    → 6
math.lcm(4, 6)      → 12   # 3.9+
math.isqrt(17)      → 4    # exact, no float
```

Floats: the big gotchas

```
0.1 + 0.2           → 0.30000000000000004
0.1 + 0.2 == 0.3    → False
math.isclose(a, b)  # compare like this
  abs_tol=1e-9      # needed near 0
round(2.5)          → 2   # ties go to even
round(3.5)          → 4
round(2.675, 2)     → 2.67
math.isnan(x)       # nan == nan is False
1e308 * 10          → inf
2.0 ** 10000        → OverflowError
math.fsum([0.1] * 10)  → 1.0
```

Floats are binary approximations. Never use them for money.

Rounding & formatting

```
int(-2.7)         → -2   # toward 0
math.floor(-2.7)  → -3   # toward -∞
math.ceil(-2.7)   → -2   # toward +∞
round(-2.7)       → -3   # nearest

f"{3.14159:.2f}"       → '3.14'
f"{1234567.891:,.2f}"  → '1,234,567.89'
f"{0.256:.1%}"         → '25.6%'
f"{12345.6789:e}"      → '1.234568e+04'
f"{10:08b}"  f"{255:#x}"  f"{10**6:,}"
```

Formatting changes the display only. Round at the end of a calculation, not in the middle.

Decimal & Fraction (exact)

```
from decimal import Decimal
Decimal("0.1") + Decimal("0.2")
                    → Decimal('0.3')
Decimal(0.1)        # bad: build from a string
Decimal("1") + 0.5  → TypeError
Decimal("2.665").quantize(Decimal("0.01"),
    rounding=ROUND_HALF_UP)  → 2.67

from fractions import Fraction
Fraction(1, 3) + Fraction(1, 6)
                    → Fraction(1, 2)
```

Decimal for money. Fraction for exact ratios. Never mix Decimal with float.

complex & cmath

```
(1+2j) * (3-1j)   → (5+5j)
z.real  z.imag  abs(z)  z.conjugate()
cmath.sqrt(-1)    → 1j
math.sqrt(-1)     → ValueError
(1+2j) < (2+3j)   → TypeError
```

The imaginary unit is `j`. Complex numbers can't be ordered.

math module

```
consts   pi tau e inf nan
powers   sqrt isqrt hypot dist exp
logs     log log2 log10
rounding floor ceil trunc
integers gcd lcm factorial comb perm prod
floats   isclose isnan isinf fsum
trig     sin cos tan  radians() degrees()
```

Trig works in radians: `math.sin(math.radians(90))`. Domain errors raise `ValueError`.

random & statistics

```
random.randint(1, 6)    # 1..6 inclusive
random.randrange(1, 6)  # 1..5
random.seed(42)         # reproducible
random.choice  sample  shuffle  uniform
secrets.token_hex()     # for security
statistics.mean  median  mode  stdev
```

BOOLEANS & LOGIC

Truthiness

```
falsy   False None 0 0.0 0j "" [] () {}
        set() range(0) Decimal(0)
truthy  "False" "0" " " [0] nan -1
isinstance(True, int)   → True
True + True             → 2
```

Python checks `__bool__`, then `__len__`.

and / or / not

```
0 or "default"  → 'default' # 1st truthy
3 and 4         → 4         # last, all true
[] and 5        → []        # 1st falsy
not x           # always True or False
all([])         → True
any([])         → False
(a > 3) or 1/0 # short-circuits, no error
1 or 2 and 3    → 1         # not > and > or
```

Idiomatic conditions

```
if items:        # not len(items) > 0
if x is None:    # not x == None
if flag:         # not flag == True
if x in (1, 2):  # not x == 1 or 2
18 <= age < 65  # chained range check
```

Comparison, identity, membership

```
==  !=  <  <=  >  >=
1 == 1.0 == True       → True
0 < x < 10            # x evaluated once
is / is not            # identity: use for None
"ell" in "hello"       → True
2 in range(0, 10, 2)   → True
1 < "a"                → TypeError
"10" < "9"             → True  # text order
(1, 2, 3) < (1, 2, 4)  → True
```

OPERATORS

Arithmetic

```
+  -  *  /  //  %  **  @      -x  +x
5 / 2         → 2.5    4 / 2   → 2.0
-7 // 2       → -4     # floors
-7 % 3        → 2      7 % -3  → -2
2 ** 3 ** 2   → 512    # right-assoc
-2 ** 2       → -4
2 ** -1       → 0.5    0 ** 0  → 1
"ab" * 3      → 'ababab'
```

`a == (a // b) * b + a % b`. The sign of `%` follows the divisor. Types: int+float → float, bool+bool → int, Decimal+float → TypeError, Fraction+float → float.

Bitwise

```
5 & 3     → 1      5 | 3     → 7
5 ^ 3     → 6      ~5        → -6
5 << 2    → 20     -5 >> 1   → -3
-5 & 0xFF → 251    # mask to 8 bits

test    (n >> i) & 1
set     n | (1 << i)
clear   n & ~(1 << i)
toggle  n ^ (1 << i)
odd     n & 1
pow2    n > 0 and n & (n - 1) == 0
lowbit  n & -n
flags   perm |= WRITE;  perm &= ~WRITE
```

`~x == -x - 1`. Python ints are unbounded, so mask after `~` or `<<` for fixed widths.

Precedence (high to low)

```
( )  x[i]  x.attr  f()
**                      # right to left
+x  -x  ~x
*  @  /  //  %
+  -
<<  >>
&
^
|
== != < <= > >= is in  # chained
not
and
or
a if c else b
lambda
:=
```

Left to right, except `**`. Unlike C, Python's bitwise operators bind tighter than comparisons: `5 & 3 == 1` is True. When in doubt, add parentheses.

Assignment & walrus

```
a = b = 0
a, b = b, a               # swap
first, *rest = [1, 2, 3]
x += 1   -=  *=  /=  //=  %=  **=
&=  |=  ^=  <<=  >>=
if (n := len(data)) > 10:  # parens!
"big" if a > 3 else "small"
```

`+=` rebinds numbers, strings and tuples, but mutates a list in place.

Operator overloading

```
a + b   __add__  (then __radd__)
a - b   __sub__      a * b   __mul__
a / b   __truediv__  a // b  __floordiv__
a % b   __mod__      a ** b  __pow__
-a  __neg__   abs(a)  __abs__
bool(a)  __bool__
==  __eq__   <  __lt__  (+ @total_ordering)
&  __and__   |  __or__   ^  __xor__

def __add__(self, other):
    if not isinstance(other, Money):
        return NotImplemented
    return Money(self.cents + other.cents)
```

Return `NotImplemented`, don't raise. Defining `__eq__` without `__hash__` makes instances unhashable.

GOTCHAS

Float equality

```
SURPRISE 0.1 + 0.2 == 0.3
FIX math.isclose(0.1 + 0.2, 0.3)
```

// rounds down, not toward zero

```
SURPRISE -7 // 2    # -4
FIX int(-7 / 2)    # -3
```

Always-true condition

```
SURPRISE if x == 1 or 2:
FIX if x in (1, 2):
```

Power vs unary minus

```
SURPRISE -2 ** 2    # -4
FIX (-2) ** 2    # 4
```

Ties round to even

```
SURPRISE round(2.5)    # 2
FIX math.floor(x + 0.5)    # 3, for x >= 0
```

is compares identity

```
SURPRISE if x is 1000:
FIX if x == 1000:
```

Strings are truthy

```
SURPRISE bool("False")    # True
FIX s.lower() in {"true", "yes", "1"}
```

Decimal from a float

```
SURPRISE Decimal(0.1)    # 0.1000000000000000055...
FIX Decimal("0.1")
```

Walrus needs parentheses

```
SURPRISE if n := len(x) > 10:    # n is a bool
FIX if (n := len(x)) > 10:
```

nan breaks sorting

```
SURPRISE sorted([3, nan, 1, 2])    # unsorted
FIX [v for v in xs if not math.isnan(v)]
```

Radians, not degrees

```
SURPRISE math.sin(90)    # 0.8939966636005579
FIX math.sin(math.radians(90))    # 1.0
```

/ returns a float

```
SURPRISE items[len(items) / 2]    # TypeError
FIX items[len(items) // 2]
```

Errors you'll meet

ZeroDivisionError

`x / 0` `x // 0` `x % 0`

TypeError

`3 + "4"` `1 < "a"` `Decimal + float`

ValueError

`int("3.5")` `math.sqrt(-1)` `int(float("nan"))`

OverflowError

`2.0 ** 10000` `int(float("inf"))`

Version notes: math.lcm 3.9 · int.bit_count 3.10 · int↔str 4300-digit limit 3.11 · sum() float accuracy 3.12 · ~ on bool deprecated 3.12.